# Week 5 Exercises: Mobile Device Features and Permissions

## Learning Objectives

After completing these exercises, you should be able to:

• Use Expo APIs to access mobile platform and device features.
• Open other applications and platform services using Linking.
• Read data from a device sensor.
• Understand continuous sensor data and subscriptions.
• Clean up subscriptions when a component is no longer active.
• Understand why mobile applications require permissions for protected features and data.
• Request and check runtime permissions.
• Handle both granted and denied permissions.
• Access the device location using Expo Location.
• Understand the difference between Expo Go, application configuration and runtime permissions.
• Test device-specific functionality on a physical device or emulator.

You may continue developing your application from the previous weeks or create a separate application for these exercises.

Using a physical Android or iOS device with Expo Go is recommended for these exercises.

## Part 1: Mobile Device Features and Expo

React Native applications can interact with features provided by the mobile operating system and the physical device.

Examples include:

• Opening another application.
• Accessing location information.
• Reading motion sensors.
• Using the camera.
• Accessing photos.
• Using haptic feedback.

Expo provides libraries that allow JavaScript applications to access many of these native capabilities.

Answer briefly:

1. What is the purpose of Expo SDK libraries in a React Native application?
2. Give three examples of device or platform features that a mobile application could access.
3. Why might some device features work differently on a physical device and an emulator?
4. Do all device features require user permission? Explain briefly.

# Part 2: Opening Other Applications with Linking

Mobile applications often need to interact with other applications installed on the device.

For example, an application may ask the operating system to:

• Open the phone application.
• Create an SMS message.
• Open an email application.
• Open a website in the browser.

Install Expo Linking if it is not already available in your project:

npx expo install expo-linking

Import the library:

import * as Linking from 'expo-linking';

Create a new screen for experimenting with Linking.

For example:

LinkingScreen

Use React Native Paper components to create a simple contact or information card.

## 2.1 Implement Linking Actions

Implement at least three different actions.

For example:

• Open the phone/dialer application using a tel: URL.
• Open the messaging application using an sms: URL.
• Open a website using an https: URL.
• Optionally open an email application using a mailto: URL.

Example:

Linking.openURL('tel:+358401234567');

Use React Native Paper Buttons or IconButtons to trigger the actions.

## 2.2 Test the Actions

Test the functionality on a physical device if possible.

Verify which application opens for each action.

Answer briefly:

1. Does your React Native application make the phone call itself?
2. What is the role of the operating system when Linking.openURL() is used?
3. What does the tel: prefix mean?
4. What is the difference between tel:, sms:, mailto: and https:?
5. Did these actions require a runtime permission from the user?

Think about the following flow:

React Native application

→ Linking

→ operating system

→ another application

# Part 3: Device Sensors

Modern mobile devices contain several sensors.

Examples include:

• Accelerometer
• Gyroscope
• Magnetometer
• Barometer

In this exercise, use the accelerometer to read information about the movement and orientation of the device.

Install Expo Sensors:

npx expo install expo-sensors

Import the accelerometer:

import { Accelerometer } from 'expo-sensors';

## 3.1 Display Accelerometer Data

Create a new screen, for example:

SensorsScreen

Store the accelerometer values in React state.

Display the following values:

• X
• Y
• Z

Use suitable React Native Paper components such as Card and Text.

The values should update automatically when the device moves.

For example, you may display values similar to:

X: 0.12

Y: -0.85

Z: 0.47

You may round the values to make them easier to read.

## 3.2 Experiment with the Sensor

Run the application on a physical device if possible.

Try:

• Holding the phone flat.
• Holding the phone vertically.
• Rotating the phone.
• Moving the phone slowly.

Observe how the sensor values change.

Do not shake or move the device in an unsafe way.

## 3.3 Sensor Subscription

The accelerometer produces data continuously.

Use Accelerometer.addListener() to subscribe to sensor updates.

Manage the subscription inside useEffect.

When the component is removed, remove the subscription.

Conceptually, your application should behave like this:

Screen opens

→ subscribe to sensor

→ receive data

→ receive data

→ receive data

→ screen closes

→ remove subscription

Answer briefly:

1. How is sensor data different from a REST API request?
2. What does a sensor subscription mean?
3. Why should the subscription be removed when it is no longer needed?
4. What is the purpose of the cleanup function returned from useEffect?
5. Did the accelerometer require a runtime permission in your application?

# Part 4: Understanding Mobile Permissions

Some mobile features provide access to sensitive information or protected hardware.

Examples may include:

• Location
• Camera
• Microphone
• Photos

The operating system controls access to these features.

Before implementing location functionality, answer briefly:

1. Why should an application not automatically have access to the user's location?
2. Who displays the system permission dialog: React Native Paper, Expo, your React component or the operating system?
3. What should an application do if the user denies a permission?
4. Is a denied permission necessarily an application error?
5. Why should applications request permissions only when they are needed?

Consider the following application states:

Permission not requested

→ Request permission

→ Granted OR Denied

If permission is granted, the application can continue using the protected feature.

If permission is denied, the application should provide appropriate feedback instead of crashing.

# Part 5: Location and Runtime Permissions

Install Expo Location:

npx expo install expo-location

Import it into your application:

import * as Location from 'expo-location';

Create a new screen, for example:

LocationScreen

The screen should initially display a message such as:

Location not requested yet.

Add a React Native Paper Button:

Use my location

Do not automatically request location permission when the screen opens. Request it when the user presses the button.

## 5.1 Request Permission

Use:

Location.requestForegroundPermissionsAsync()

Check the result before accessing the location.

Your application must handle at least these two cases:

Permission granted

→ Continue to retrieve the location.

Permission denied

→ Display an appropriate message.

For example:

Location permission denied.

The application must not crash if the user denies the permission.

### 5.2 Retrieve the Current Location

If permission is granted, retrieve the current position using Expo Location.

Display at least:

• Latitude
• Longitude

Use React state to store the location.

You may also display an appropriate message while the application is retrieving the location.

For example:

Getting location...

and after a successful result:

Location received.

### 5.3 Test Both Permission States

Do not only test the successful case.

Test what happens when:

1. Location permission has not yet been requested.
2. The user grants permission.
3. The user denies permission.

Observe what happens if you try to request the permission again after denying it.

You may also examine values such as:

• status
• granted
• canAskAgain

Note: The exact permission behaviour may differ between Android and iOS and may also depend on previous permission choices. You may need to change the permission from the device settings or reset the application state when testing.

## Part 6: Expo Go, Permissions and Application Configuration

During development, Expo Go already contains native functionality for many Expo SDK libraries.

This makes it possible to test many device features without first building your own native application.

However, there is an important difference between:

• Native application configuration.
• Runtime permission requests.

Runtime permission means that the running application asks the user for access to a protected feature.

For example:

Location.requestForegroundPermissionsAsync()

Native application configuration defines which capabilities and permission descriptions are included when your own Android or iOS application is built.

Answer briefly:

1. What is the difference between application configuration and a runtime permission request?
2. Why is Expo Go convenient when learning and testing device APIs?
3. Why should you still check whether a particular Expo library is supported in Expo Go?
4. Why might your own production build require additional configuration even if the feature worked during development in Expo Go?

# Part 7: Small Extension

Choose ONE small extension for your application (previous week's exercise work).

Examples:

• Add an email action to your Linking screen.
• Add a button for starting and stopping accelerometer updates.
• Experiment with another sensor available through Expo Sensors.
• Display additional information returned by Expo Location.
• Add a button that opens a map application or map URL using the retrieved location.
• Improve the UI and clearly show the current state of a device feature.

Keep the extension small. The purpose is to experiment with the device APIs rather than build a large new feature.

# Part 9: Reflection

Answer briefly:

1. What was the most important difference between Linking, Sensors and Location?
2. Which feature required a runtime permission and why?
3. What should an application do when a user denies a permission?
4. What is the difference between a one-time asynchronous operation and a subscription?
5. Why is useEffect cleanup important when using subscriptions?
6. What role does the operating system have when Linking is used?

7. What role does Expo have between your JavaScript code and native device functionality?
8. What was different when testing these features on a physical device compared with web development?
9. How did you use AI-assisted development tools, if any? Describe one suggestion that you tested or verified yourself.

# Submission

Submit:

• Your application source code.
• A screenshot of your Linking screen.
• A screenshot showing accelerometer data.
• A screenshot of your Location screen.
• Evidence that your application handles a denied location permission.
• Your short written answers.